
Week Four: Regular Languages; Pumping Lemma

Proposition. The class of languages accepted by DFA's are closed under the following operations:

- (1) union, concatenation, Kleene star;
- (2) complementation;
- (3) intersection.

Note: we shall present two different constructions for intersection. The first will be somewhat implicit and makes use of (2) while the second construction will use the cross product of automata.

Let $M_1 = (Q_1, \Sigma, \delta_1, s_1, F_1)$ and $M_2 = (Q_2, \Sigma, \delta_2, s_2, F_2)$. Then we let $M = (Q_1 \times Q_2, \Sigma, \delta, (s_1, s_2), F)$ where $\delta((q, q'), a) = (\delta_1(q, a), \delta_2(q', a))$, $q \in Q_1, q' \in Q_2, a \in \Sigma$, and $F = F_1 \times F_2 = \{(f, f') : f \in F_1, f' \in F_2\}$.

The idea of the cross product construction is that a computation in M actually will trace out the computation in the two machines M_1 and M_2 simultaneously.

Note: if the collection of accepting states is changed to $(F_1 \times Q_2) \cup (Q_1 \times F_2)$, then M would accept the union $L(M_1) \cup L(M_2)$. So, this construction will produce a DFA that accepts the union rather than a NFA.

The regular languages are closed under homomorphisms.

A *homomorphism* is a map $h : \Sigma^* \rightarrow \Gamma^*$ such that for all $x, y \in \Sigma^*$ we have that

$$h(xy) = h(x)h(y) \text{ and } h(\epsilon) = \epsilon.$$

It follows at once that the values of h are determined on any string once they are known for the letters of Σ^* .

Proposition. Let $h : \Sigma^* \rightarrow \Gamma^*$ be a homomorphism. Let $L \subseteq \Sigma^*$ be a regular language. Then $h(L)$ is also regular.

Sketch of Proof: We use the same technique as for the reversal of a regular language as well as the construction of a NFA from a regular expression.

We use the fact that L is denoted by some regular expression. Then the argument reduces to the following formula:

$$L(h(\beta)) = h(L(\beta)),$$

for any regular expression β . This formula can be established by induction on the number of operators in the regular expression β .

The base case of zero operators corresponds to $\beta = \sigma \in \Sigma, \epsilon$ or $\emptyset$. In all three cases, the desired formula holds.

We next need to make the observations that

$$(1) \ h(L_1 \circ L_2) = h(L_1) \circ h(L_2),$$

$$(2) \ h(\bigcup L_k) = \bigcup h(L_k).$$

$$(3) \ h(L^*) = h(L)^*.$$

The result now follows in a routine fashion.

If L is any language, let $\text{Prefix}(L)$ denote the set of all prefixes of strings in L ; that is,

$$\text{Prefix}(L) = \{x : xy \in L\}.$$

Proposition: The set of prefixes of a regular language is regular.

Proof: Let $L = L(M)$ where $M = (Q, \Sigma, \delta, s, F)$ be a DFA accepting L . Without loss of generality, we assume all the states of M are reachable from the start state.

To show that $\text{Prefix}(L)$ is regular, we find a DFA $\widehat{M}$ that will accept it.

We find that $\widehat{M} = (Q, \Sigma, \delta, s, \hat{F})$; that is, the DFA that accepts the set of prefixes is the same as the one for L except we only change the set of accepting states.

We say that $q \in \hat{F}$ if and only if there is a path from q to an accepting state f of the original machine M .

In other words, there is a string $y \in \Sigma^*$ so that $\delta(q, y) = f \in F$.

This works because: if $\delta(s, x) = q$, then $x \in \text{Prefix}(L)$ since $\delta(s, xy) = f \in L$; and conversely.

Informally, the set of accepting states for $\text{Prefix}(L)$ consist of any state which can be joined to an accepting state of the original machine.

We now consider a generalization of the set of prefixes.

Let L_1 and L_2 be languages over Σ . Then the *right quotient* of L_1 with L_2 is defined as

$$L_1/L_2 = \{x : xy \in L_1 \text{ for some } y \in L_2\}.$$

Example: $L/\Sigma^* = \text{Prefix}(L)$ while $L/\{\epsilon\} = L$.

Proposition: The L_1 and L_2 are regular languages over Σ . Then their right quotient L_1/L_2 is regular. That is, the family of regular languages is closed under right quotient with a regular language.

Proof: The argument is a generalization of the one for prefixes.

Let $L_1 = L(M_0)$ where $M = (Q, \Sigma, \delta, q_0, F)$ be a DFA accepting L_1 . Without loss of generality, we assume all the states of M are reachable from the start state.

To show that L_1 is regular, we find a DFA $\widehat{M}$ that will accept it.

We find that $\widehat{M} = (Q, \Sigma, \delta, q_0, \hat{F})$; that is, the DFA that accepts the right quotient L_1/L_2 is the same as the one for L_1 except we only change the set of accepting states.

We say that $q \in \hat{F}$ if and only if there is a string $y \in L_2$ so that $\delta(q, y) = f \in F$.

This works because: if $\delta(s, x) = q$, then $x \in L_1/L_2$ since $\delta(s, xy) = f \in L$; and conversely.

The construction will be complete when we indicate how to recognize such states q .

Let $M_j = (Q, \Sigma, \delta, q_j, F)$ so M_j is the same DFA as M except for the start state. Note: $M = M_0$.

We find $q_j \in \hat{F}$ if and only if $L(M_j) \cap L_2 \neq \emptyset$ for if this intersection is non-empty it must contain some string, say y . But $\delta(q_j, y) = f \in F$ which is the original condition given above.

bf Example: Find L_1/L_2 where $L_1 = L(a^*baa^*)$ and $L_2 = L(aba^*)$.

DFA Transitions (4d)		
State	a	b
q_0	q_0	q_1
q_1	q_2	q_3
q_2	q_2	q_3
q_3	q_3	q_3

start state: 0, accepting state: 2

The DFA for $\text{Prefix}(L_1)$ has the same transition graph as for L_1 itself with start state q_0 and accepting states q_0, q_1, q_2 .

Next, let M_j be the DFA whose transition graph is the same as the DFA for L_1 with the same accepting states but whose start state is q_j , with $j = 0, 1, 2, 3$.

We find $L(M_0) = L_1$, $L(M_1) = L(aa^*)$, $L(M_2) = L(a^*)$, and $L(M_3) = \emptyset$.

The DFA for the right quotient L_1/L_2 has the same transition graph as for L_1 with start state q_0 and accepting state q_0 .

There are algorithms to answer the following questions about DFA's:

- (1) given $w \in \Sigma^*$, is $w \in L(M)$?
- (2) Is $L(M) = \emptyset$?
- (3) Is $L(M) = \Sigma^*$?
- (4) Given two DFA's M_1 and M_2 , is $L(M_1) \subseteq L(M_2)$?
- (5) Given two DFA's M_1 and M_2 , is $L(M_1) = L(M_2)$?

Proof. will be given in lecture.

2. We now present a useful theoretical result that states that regular languages must obey a certain type of "periodicity" property. It is used to show that certain simple languages cannot be regular.

Pumping Lemma. Let $M = (K, \Sigma, \delta, s, F)$ be a DFA, with $L = L(M)$. Suppose $m = |K|$. Let $w \in L(M)$ with $|w| \geq m$. Then there are strings x, y , and z such that

$$w = xyz, \quad |xy| \leq m, y \neq e, \quad \text{and} \quad xy^kz \in L, \quad \forall k \geq 0.$$

We call m , the *pumping constant*.

Idea of the Proof. Any string accepted by M whose length is greater than the number of states of the machine must have a loop in it. It is precisely this loop that can be iterated.

We use the contrapositive form of the pumping lemma to show that a language is NOT regular -

Let L be a language. Suppose that there exists a string w with substrings x, y, z such that $y \neq e, w = xyz$, and $xy^kz \notin L$, for some integer $k \geq 0$, then L cannot be a regular language.

So, to show a language is NOT regular, think of playing the following sort of game: find a string $w \in L$ so that for any non-empty substring y of w , there exists some pumped form of $w : xy^kz$ so that $xy^kz \notin L$.

Example (1) $L_1 = \{a^n b^n : n \geq 1\}$ is not regular.

Suppose the language were regular. Choose n greater than the pumping constant given above. Then $w = a^n b^n$ can be factored as $xyz, y \neq e$, and $xy^kz \in L_1$, for all $k \geq 0$. Choose $k = 0$, so $xz \in L$, but $xz = a^{n-|y|} b^n \in L$. Contradiction. We say that we pumped "down" in this example.

Example (2) $L_2 = \{a^{n^2} : n \geq 1\}$ is not regular.

Suppose L_2 were regular. Choose n greater than the pumping constant m . Then $a^{n^2} = xyz$, where $y \neq e$ and $|y| \leq m \leq n$. So, $xy^kz \in L_2$, for all $k \geq 0$. Choose $k = 2$. Then $xy^2z \in L_2$ implies $|xy^2z|$ is a perfect square. But $n^2 < |xy^2z| < n^2 + n < (n+1)^2$. Contradiction. In this example, we say that we pumped "up."

Example (3) $L_3 = \{w \circ w^R : w \in \Sigma^*\}$ is not regular if $\Sigma = \{a, b\}$.

Suppose L_3 were regular. Choose the string w so $|w| \geq m + 1$, where m is the pumping constant. Further, we may choose w to have the special form: $w = a^m b$, so $ww^r = a^m bba^m$. By the Pumping Lemma, $ww^R = xyz$, with $|xy| \leq m$ and $xy^kz \in L_3$, for $k \geq 0$. Take $k = 0$. Then $xz = a^{m-|y|}bba^m \in L_3$. Contradiction.

Algorithm to determine if two DFA's are equivalent

Let $M = (Q, \Sigma, \delta, s, F)$ and $M' = (Q', \Sigma, \delta', s', F')$ be two DFA's. For the sake of notation, we shall take $\Sigma = \{a, b\}$.

We form a table with and three columns ($= |\Sigma| + 1$). and at most $|Q \times Q'|$ row entries. A typical row entry is:

$$(q, q'), \quad (\delta(q, a), \delta(q', a)), \quad (\delta(q, b), \delta(q', b)), \quad \text{where } q \in Q, q' \in Q'.$$

The first row begins with the entry (s, s') . Then if either $(\delta(s, a), \delta(s', a))$ or $(\delta(s, b), \delta(s', b))$ are not found in the first column, then they are added to the first column for later expansion. If there is ever a pair (p, p') such that $p \in F$ and $p' \notin F'$ or $p \notin F$ and $p' \in F'$, we stop the table construction and conclude that the DFA's are inequivalent. If the process stops with no uncompleted rows, we conclude that the DFA's are equivalent.

Example. The first DFA has start state q_1 and the set of final states is $\{q_1\}$. Its transition table is:

State	a	b
1	1	2
2	3	1
3	4	3
4	3	1

The second DFA has start state q_1 and the set of final states is $\{q_1\}$. Its transition table is:

State	a	b
1	1	2
2	3	1
3	2	3

We form the table according to the algorithm. Note: the first pair is the pair of start states of the two machines.

Table for Equivalence of DFA's		
Pair of States (q, q')	$(\delta(q, a), \delta(q', a))$	$(\delta(q, b), \delta(q', b))$
(1, 1)	(1, 1)	(2, 2)
(2, 2)	(3, 3)	(1, 1)
(3, 3)	(2, 4)	(3, 3)
(2, 4)	(3, 3)	(1, 1)

We conclude that these two DFA's are equivalent.